Gymnasium Oberwil

Maturarbeit

Bildsynthese von menschlichen Gesichtern mithilfe von Kuenstlicher Intelligenz

Eine Arbeit ueber Kuenstliche Neuronale Netze, Convolutional Neural Networks, Autoencoders und Tensorflow

Luis Wirth, 4a

Betreut von
Dr. Jonas Gloor

Abgegeben im September 2019
LaTeX

Inhaltsverzeichnis

1 Vorwort **3**

2 Einleitung **4**

3 Konventionen und Notation **5**

4 Machinelles Lernen - Theorie **7**
4.1 Allgemeine Begriffe . . . 7
4.1.1 Daten . . . 7
4.1.2 Modelle . . . 8
4.2 Sonstiges . . . 8
4.2.1 Supervised Learning . . . 8
4.2.2 Unsupervised Learning . . . 8
4.2.3 Overfitting . . . 8
4.3 Training . . . 8
4.3.1 Kostenfunktionen . . . 8
4.3.2 Gradientenverfahren . . . 9
4.3.3 Stochastisches Gradientenverfahren fuer Machine Learning . . . 11
4.3.4 Adam Optimizer . . . 12
4.4 Künstliche Neuronale Netze und Deep Learning . . . 13
4.4.1 Perzeptron . . . 13
4.4.2 Biologische Analogie . . . 14
4.4.3 Erweiterung der Kuenstlichen Neuronen . . . 14
4.4.4 Topologie der Kuenstlichen Neuronalen Netzen . . . 16
4.4.5 Vorwaertspropagierung . . . 17
4.4.6 Gewichte und Neigungen initalisieren* . . . 19
4.4.7 Rueckwaertspropagierung . . . 19
4.4.8 Universal Approximation Theorem . . . 20
4.5 Convolutional Neural Networks . . . 21
4.5.1 Bilder als 3D-Matrizen . . . 21
4.5.2 Filter . . . 22
4.5.3 Convolutional Layer . . . 26
4.5.4 Pooling Layer . . . 26
4.5.5 Fully-connected Layer . . . 27
4.5.6 Das Netzwerk und sein Aufbau . . . 27
4.5.7 Eigenschaften von CNNs . . . 27
4.6 Autoencoder . . . 28
4.6.1 Autoencoder . . . 28
4.6.2 Bedeutung von Autoencodern . . . 28
4.6.3 Variational Autoencoder? . . . 28
4.6.4 Detangled Autoencoder? . . . 28
4.7 Programmiertools . . . 28
4.7.1 Tensorflow . . . 28

5 Entwicklung eines Autoencoders **29**
5.1 Tools . . . 29
5.1.1 Python . . . 29
5.1.2 Tensorflow . . . 29

5.1.3 Keras 29
5.2 Idee 29
5.3 Voraussetzungen 29
5.3.1 Trainingsdaten 29
5.4 Programmieren 29

6 Diskussion? 30

7 Schlusswort 31

Literatur 32

1 Vorwort

2 Einleitung

Die Leitfrage lautet: **Wie können mithilfe von Machine Learning Bilder von künstlichen meschlichen Gesichtern generiert werden?**

3 Konventionen und Notation

Beschriftung

Zahlen und Tensoren

a	ein Skalar (Zahl)
$\mathbf{a}$	ein Vektor
$\boldsymbol{A}$	eine Matrix
$\boldsymbol{A}_{n\times m}$	Matrix mit n Zeilen und m Spalten
$\overline{\mathbf{A}}$	ein Tensor
$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \cdots$	a
$\mathfrak{a}, \mathfrak{b}, \mathfrak{c}, \mathfrak{d}, \mathfrak{e}, \mathfrak{f}, \mathfrak{g}, \cdots$	a

Mengen

(a, b)	ein geordnetes Paar (2-Tupel) der Elemente a und b
$\mathbb{A}$	eine Menge
$\mathbb{R}$	die Menge aller Reeller Zahlen
$\mathbb{Z}$	die Menge aller ganzer Zahlen
$\mathbb{N}$	die Menge aller natuerlicher Zahlen
$\mathbb{E}$	die Menge aller gerader (engl.: even) Zahlen
$\mathbb{O}$	die Menge aller ungerader (engl.: odd) Zahlen
$\{a, b\}$	die Menge, welche aus Elementen a und b besteht
$\{1, \ldots, n\}$	Menge aller ganzen Zahlen von 1 bis n

Indexierung

a_i	i-te Vektorkomponente mit Startindex 1
$\boldsymbol{A}_{n\times m}$	Matrix mit n Zeilen und m Spalten
$(\boldsymbol{A})_{i,j} =$	Matrixelement in der i-ten Zeile und der j-ten Spalte
$A_{i,j}$	Matrixelement in der i-ten Zeile und der j-ten Spalte
$\boldsymbol{A}_{i,:}$	Zeile i der Matrix $\boldsymbol{A}$
$\boldsymbol{A}_{:,i}$	Spalte i der Matrix $\boldsymbol{A}$

Lineare Algebra Operationen

$\boldsymbol{A}^{\mathrm{T}}$	das Transponierte einer Matrix $\boldsymbol{A}$
$\boldsymbol{A} \odot \boldsymbol{B}$	Elementeweises (Hadamard) Produkt

Infinitesimalrechnung/Differentialrechnung

$\frac{\mathrm{d}y}{\mathrm{d}x}$	Ableitung von y bezueglich x
$\frac{\partial y}{\partial x}$	Partielle Ableitung von y bezueglich x
∇y	Gradient von y
$\nabla_{\mathbf{x}} y$	Gradient von y bezueglich $\mathbf{x}$ (Vektor)

Funktionen

$f : \mathbb{A} \to \mathbb{B}$	Funktion f mit Definitionsmenge $\mathbb{A}$ und Zielmenge $\mathbb{B}$
$f(x)$	Funktion f mit Argument x (Skalar)
$f(\mathbf{v})$	Funktion f mit Argument $\mathbf{v}$ (Vektor)
$f[\mathbf{v}]$	Vektorisierte Funktion f mit Argument $\mathbf{v}$ (Vektor)
$\mathcal{A}$	ein Funktionenraum

Machine Learning

4 Machinelles Lernen - Theorie

Der Themenbereich des **Machinellen Lernens** beschäftigt sich mit Algorithmen und mathematischen Modellen, welche von selber lernen, Probleme zu loesen. Hierzu werden keine expliziten Instruktionen fest einprogrammiert (engl.: hard coding), sondern das Modell wird trainiert und optimiert sich von selbst. Dabei wird das Modell mit Trainingsdaten befuettert, mithilfe dessen eine Korrelation zwischen den Inputdaten und den Outputdaten erlernt werden soll. Der Zusammenhang der Daten wird von den Modellen verallgemeinert und das Modell kann dann Vorhersagen fuer neue Daten machen.

4.1 Allgemeine Begriffe

4.1.1 Daten

Um ein Modell zu trainieren braucht man Daten. Diese Daten kommen in der Form eines **Trainingsdatensatz** $\mathbb{X}$. Dabei handelt es sich um eine Menge von **Trainingssampeln**, welche jeweils aus **Inputs** und gewuenschten Outputs. bestehen. Diese gewuenschten Outputs bezeichnet man auch als **Labels**. Diese Inputs werden in einem Vektor $\mathbf{x} = (x_1, x_2, \ldots, x_n)^{\mathrm{T}}$ und die Labels in einem Vektor $\hat{\mathbf{y}} = (\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_m)^{\mathrm{T}}$ zusammengefasst. Die Labels sind zu unterscheiden von den vorhergesagten **Outputs**, welche der Algorithmus produziert. Auch diese Outputs werden in einem Vektor $\mathbf{y} = (y_1, y_2, \ldots, y_m)^{\mathrm{T}}$ zusammengefasst. Somit ist ein Trainingssample nichts anderes als ein Paar $(\mathbf{x}_i, \mathbf{y}_i)$ eines Inputsvektor $\mathbf{x}$ und eines Labelvektor $\hat{\mathbf{y}}$.

Die Inputs bestehen aus gewissen **Attributen**, welche die Merkmals*typen* der Daten sind. Die sogennanten **Features** sind dann die spezifischen Werte der Attribute eines Samples. Der Algorithmus soll anhand dieser Attribute seine Vorhersagen machen. Diese Vorhersagen werden dann mit den Labels abgeglichen und so bewertet. Anhand der Bewertungen wird dann eine Optimierung des Modells vorgenommen. Unter korrekten Bedingungen (keine Ueberanpassung) findet kein Auswendiglernen der Trainingsdaten statt, sondern ein Generalisieren des Zusammenhangs anhand von Mustern und Gesetzmassigkeiten.

Um eine endgueltige Bewertung des Modells durchzufuehren, wird ein Testdatensatz $\mathbb{T}$, welcher nicht Teil des Trainingsdatensatz ist, genutzt um Vorhersagen zu machen. Dies garantiert, dass kein Auswendiglernen moeglich ist.

BESSERS BEISPIEL -> UM SPAETER KONKRETES MODELL ZU ERKLAEREN AUTOPREIS SCHAETZEN (OKASSION) ANHAND VON GEFAHRENEN KILOMETERN Am Beispiel eines Datensatz, welcher fuer Bilderkennung von Hunden und Katzen genutzt wird, werden diese Begriffe verdeutlicht. Die Inputdaten sind die Fotos von Hunden und Katzen. Deren Attribute sind die moeglichen RGB-Farbenwerte (drei Zahlen, welche von 0 bis 255 reichen) aller Pixel. Die Features sind die spezifischen RGB-Werte der Pixel fuer die einzelnen Bilder. Die Outputdaten sind die Labels, welche in diesem Fall angeben, ob ein Hund oder eine Katze auf dem Bild gezeigt wird. Dies wird meistens mit einer One-Hot-Kodierung bzw. einem One-Hot-Vektor gemacht. Jeder Labelart wird eine eigene Vektorkomponente zugewiesen. Falls das Label aktiv ist, ist die Komponent 1 ansonsten 0. Somit kann Hund als $\mathbf{l}_H = (1, 0)^T$ reprasentiert werden und Katze als $\mathbf{l}_K = (0, 1)^T$.

Abbildung 1: das beschriebene Machine Learning Modells

4.1.2 Modelle

Ein **Modell** ist eine mathematische Funktion $f\colon \mathbb{R}^n \to \mathbb{R}^m$, welche die Inputs auf die Outputs abbildet $\mathbf{y} = f(\mathbf{x})$. Im Zusammenhang mit Wertevorhersagen spricht man von einem Regressionsmodell. Das Verhalten dieses Modells wird bestimmt durch seine **Modellparameter** $\lambda_1, \lambda_2, \ldots$. Das Ziel ist es, die Modellparameter so einzustellen, dass die Vorhersagen $\mathbf{y}$ besser mit den Labels $\hat{\mathbf{y}}$ uebereinstimmen. Dies wird iterativ gemacht, indem immer wieder leichte Anpassungen an den Parametern vorgenommen werden, bis das Modell die gewuenschten Resultate liefert.

Das wohl einfachste Regressionsmodell ist eine Regressionsgerade. Diese ist angemessen, falls ein einfacher linearen Zusammenhang der Form $y = \lambda_1 x + \lambda_0$ zwischen den Features und den Labels besteht. Nun muessten nur noch λ_0 und λ_1 bestimmt werden.
Neben den gelernten Parametern, gibt es auch noch sogenannte **Hyperparameter**. Diese koennen nicht erlernt werden, sondern muessen manuell vor dem Training gewaehlt werden und koennen den Lernvorgang erheblich beeinflussen.

Fuer Machine Learning haben sich gewisse Modelle besonders guet etabliert, darunter waeren: Support Vector Machines, Evolutionäre Algorithmen, und Künstliche Neuronale Netze. Diese Arbeit wird sich vorwiegend mit Neuronalen Netzen auseinandersetzen.

BEISPIELMODELL GLEICHES WIE VORHIN

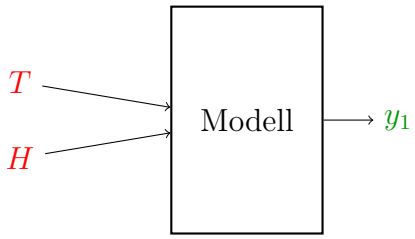

Abbildung 2: beschriebenes Beispielmodells

4.2 Sonstiges

4.2.1 Supervised Learning

4.2.2 Unsupervised Learning

4.2.3 Overfitting

4.3 Training

4.3.1 Kostenfunktionen

Einsicht ist der erste Schritt zur Besserung. Das gilt auch beim Machine Learning, deshalb muss beim Training zuerst die Genauigkeit des Modells bewertet werden. Dafuer gibt es sogennante **Kosten-, Verlust- oder Fehlerfunktionen**. Sie sollen ein Mass fuer die Abweichung des Outputs $\mathbf{y}$ vom Label $\hat{\mathbf{y}}$ sein.

Die Fehler der einzelnen Outputs $c(y_i, \hat{y}_i)$ werden aufsummiert um den Fehler $C(\mathbf{y}_j, \hat{\mathbf{y}}_j)$ (wobei $\mathbf{y}_j$ die Vorhersagen sind und $\hat{\mathbf{y}}_j$ die Labels sind) einer gesamten Vorhersage zu berechnen. Siehe Gleichung (1).
Der Fehler $\bar{C}(\mathbb{X})$ eines ganzen Datensatzes $\mathbb{X}$, der Groesse p ergibt sich aus dem arithmetischen Mittel der einzelnen Vorhersagen-Fehler. Siehe Gleichung (2).

$$C\left(\mathbf{y}, \hat{\mathbf{y}}\right) = \sum_{i=1}^{m} c(y_i, \hat{y}_i) \qquad (1) \qquad\qquad \bar{C}(\mathbb{X}) = \frac{1}{p}\sum_{j=1}^{p} C\left(\mathbf{y}_j, \hat{\mathbf{y}}_j\right) \qquad (2)$$

Eine Kostenfunktion sollte folgende Eigenschaften aufweisen:

- c ist minimal, wenn $y = \hat{y}$
- c waechst mit $|\mathbf{y} - \hat{\mathbf{y}}|$
- C ist nach jedem y_n partiell differenzierbar (erklaert in Sektion (**??**))

Eine beliebte Kostenfunktion ist die "Mittlere quadratische Abweichung" (engl.: mean squared error).
$C_{MSE} = \frac{1}{2n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i\right)^2 = \frac{1}{2n}(\hat{\mathbf{y}} - \mathbf{y})^2$. Sie erfuellt alle anforderungen, denn: Sie ist 0 falls $y = \hat{y}$. Sie ist proportional zu $\left(\hat{y} - y\right)^2$ und ihre Ableitung nach y lautet: $C' = \frac{1}{n}(y - \hat{y})$

4.3.2 Gradientenverfahren

Um ein Modell nun zu trainieren, muss man verstehen, dass es sich um ein Optimierungsproblem handelt. Das Modell ist am besten, also macht die besten Vorraussagen, wenn die Funktionswerte der Fehlerfunktion am kleinsten sind. Deshalb muss diese Fehlerfunktion C minimiert werden. Hierbei muss die Funktion C nicht mehr in Abhaengigkeit der Inputs betrachtet werden, sondern in Abhaengigkeit von den Modellparametern $C(\lambda_1, \lambda_2, \ldots, \lambda_n)$, denn diese sollen angepasst werden, um das Modell zu verbessern. Fuer diese Optimiertung wird das sogennante **Gradientenverfahren** (engl.: Gradient descent) verwendet. [1]

Allgemein koennen mithilfe des Gradientenverfahrens Funkionen $f(x_1, x_2, \ldots, x_n)$ vom Typ $\mathbb{R}^n \to \mathbb{R}$ (wie z.B. die Fehlerfunktion) minimiert werden. Dies geschieht, indem ein Startpunkt (Ortsvektor) $\mathbf{p}_0$, dessen Komponenten den Input von f darstellt, gewaehlt wird. Nun werden iterativ neue Punkte $\mathbf{p_z}$ gesucht, welche immer naeher beim lokalen Minimun liegen, also Punkte, die den Funktionswert $f(\mathbf{p}_z)$ immer kleiner werden lassen. Dies wird durchgeführt, bis der Punkt genug nahe beim lokalen Minimun ist.

Dafuer muss ein Vektor $\mathbf{b}_z$ bestimmt werden, welcher auf den Punkt $\mathbf{p}_z$ draufaddiert einen neuen Punkt $\mathbf{p}_{z+1}$ bildet, bei dem der Funktionswert $f(\mathbf{p}_{z+1})$ kleiner ist als der von $f(\mathbf{p}_z)$. Dies geschieht am effizientesten, wenn $\mathbf{b}_z$ in die Richtung der staerksten Funktionswertabnahme zeigt.
Hierzu braucht man den sogennanten **Gradient** ∇, wofür man wiederum partielle Ableitungen braucht.

[1] Im Gymnasium wird beigebracht die lokalen Extrema (inkl. lokalen Minimas) zu bestimmen, indem die erste Ableitung f' gebildet wird und gleich null gesetzt wird. Dies ist hier nicht moeglich, da die Funktion $C'(\lambda_1, \lambda_2, \ldots, \lambda_n)$ deutlich zu komplex ist, um die Nullstellen zu bestimmen. Deshalb wird das Gradientenverfahren verwendet.

Partielle Ableitungen

Partielle Ableitungen sind eine Erweiterung der "normalen" Ableitungen auf multidimensionale Funktionen $f(x_1, \ldots, x_n)$. Man leitet dabei nur nach einem Parameter x_i ab und betrachtet die restlichen Argumente als Konstanten. Es gelten die gleichen Ableitungsregeln wie bei der nicht-partiellen Ableitung. Die partielle Ableitung einer Funktion $f(x_1, \ldots, x_n)$ bezueglich der Variable x_i in einem Punkt $\mathbf{a} = (a_1, \ldots, a_n)^{\mathrm{T}}$ ist analog zur "normalen" Ableitung folgendermassen definiert:

$$\frac{\partial f}{\partial x_i}(\mathbf{a}) := \lim_{h \to 0} \frac{f(a_1, \ldots, a_i + h, \ldots, a_n) - f(a_1, \ldots, a_i, \ldots, a_n)}{h}$$

Geometisch ist dies die Steigung der Tangente im Punkt $\mathbf{a}$ in Richtung der Achse des Parametern x_i, nach dem man ableitet.

Gradient

Der Gradient ∇ ist ein Differentialoperator, welcher ein Skalarfeld auf ein Vektorfeld (das sogennante Gradientenfeld) abbildet.
Dabei fasst man alle partiellen Ableitungen einer Funktion f in einem Vektor zusammen.

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Geometisch ist der Gradient $\nabla f(\mathbf{p})$ einer Funktion f in einem Punkt $\mathbf{p}$ dann der Vektor, welcher in die Richtung des steilsten Anstiegs von f zeigt. Sein Betrag gibt die Staerke des Anstiegs an.

Da der Gradient in die Richtung des steilsten Anstiegs zeigt, sollte der Vektor $\mathbf{b}_z$ in die Richtung des negierten Gradient der Funktion f im Punkt $\mathbf{p}_z$ zeigen. Also kann jetzt das iterative Annahern an das lokalen Minimums so beschrieben werden (wobei η eine spaeter erklaerte Schrittgroesse darstellt):

$$\mathbf{p}_{z+1} = \mathbf{p}_z - \eta \cdot \nabla f(\mathbf{p}_z) \tag{3}$$

BEISPIEL FUER GRADIENTENVERFAHREN MIT ABBILDUNG

Abbildung 3: Visualisierung des Gradientenabstieg

Waehrend des Gradientenverfahren konvergiert der Punkt $\mathbf{p}_z$ zu einem belibigen *lokalen* Minimum, abhängig davon wie der Startpunkt $\mathbf{p}_0$ gewaehlt wurde. Da dieser meist zufaellig bestimmt wird, ist es eine Glückssache ein sehr niedriges Minimum zu finden.

Die sogennante **Lernrate** η aus Gleichung (3) ist ein Hyperparameter. Sie ist ein positiver Proportionalitatsfaktor, welcher die Schrittgroesse des Gradientenabstieg bestimmt. Sie muss je nach zu minimierender Funktion anderst gewaehlt werden. Dabei hilft nur ausporbieren. Falls η nicht gut gewahelt wurde, gibt es Probleme beim Training:

- Falls η zu klein ist, verlaueft das Trainings unnotig langsam und braucht sehr lange. Ausserdem kann es passieren, dass man bei einem hohen lokalen Minimum stecken bleibt.
- Falls η zu gross ist, passiert es, dass man ueber das lokale Minimum hinaus schiesst und nur darum herum springt.

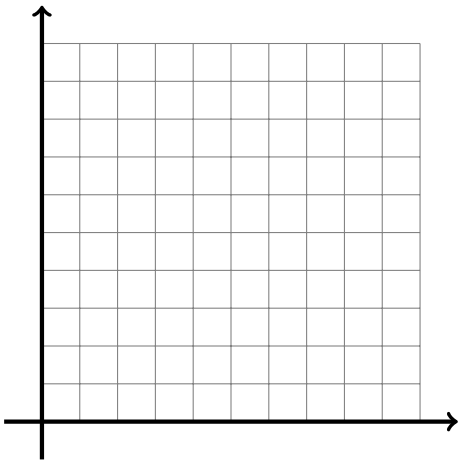

Abbildung 4: Auswirkungen verschiedener Lernraten

4.3.3 Stochastisches Gradientenverfahren fuer Machine Learning

Wie vorhin erklaert, wird fuer das Trainieren eines Modells das Gradientenverfahren benutzt. Konkret wird die Kostenfunktion $C(\mathbf{y}, \hat{\mathbf{y}}|\lambda_0, \ldots, \lambda_n)$ (λ sind die Modellparameter, $\mathbf{y}$ die Vorhersagen und $\hat{\mathbf{y}}$ die Labels) minimieren, indem die Parameter angepasst werden. Dadurch macht das Modell immer bessere Vorhersagen. Fuer nur eine Iteration des Gradientenverfahren, muesste man den Gradienten fuer den *gesamten* Trainingsdatensatz berechnen. Dies waere zwar ein exakter Prozess, aber ein extrem langsamer zugleich. Bei grossen Datensaetzen wuerde es eine Ewigkeit dauern, bis das Modell nur annahernd guete Vorhersagen machen wuerde.

Aus diesem Grund verwendet man eine leicht abgeänderte Variante dieses Verfahren, naemlich das **Stochastische Gradientenverfahren** (engl.: Stochastic Gradient Descent). Hierfuer wird der "echte" Gradient des gesamten Datensatzes mit dem Gradienten einiger Trainingsbeispiel approximiert. Dazu wird der Trainingsdatensatz in sogennante **Mini-Batches** eingeteilt und der Gradient jeweils pro Mini-Batch berechnet. Als Konsequenz finden deutlich mehr Iterationen statt in *einer* Durchkaemmung der Trainingsdaten, welche man als **Epoche** bezeichnet. Oft wird mehrere Epochen lang trainiert. Der Gradient eines genug grossen Mini-Batches ist zwar nicht ganz exakt, aber approximiert den Gradieten des gesamten Datensatzen genuegend gut. Sowohl die Mini-Batch Groesse, wie auch die Anzahl Epochen sind weitere Hyperparameter.

Die partiellen Ableitungen der gesamten Trainingsdaten wird mit dem arithmetischen Mittel der partiellen Ableitungen eines Mini-Batches der Groesse q approximiert. Siehe Gleichung (4).

$$\frac{\partial \bar{C}}{\partial \lambda_k} \approx \frac{1}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_k} \tag{4}$$

Eine Iteration des Stochastischen Graidentenverfahren wird analog zu Gleichung (3) folgendermassen durchgefuehrt:

$$\lambda_{k,t+1} = \lambda_{k,t} - \frac{\eta}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_{k,t}} \tag{5}$$

4.3.4 Adam Optimizer

ZUSAMMENFASSUNG TRAININGSALGORITHMUS

4.4 Künstliche Neuronale Netze und Deep Learning

Das wohl beste Modell fuer die meisten Problemstellungen in Bereich Machinelles Lernen (Bilderkennung, Spracherkennung, etc.) ist das **Kuenstliche Neuronale Netz**, kurz KNN (eng.: Neural Network). Die Methoden welche benutzt werden um Neuronale Netze zu traniert fasst man unter dem Begriff **Deep Learning** zusammen.
Kuenstliche Neuronale Netze sind zum Teil biologisch inspiriert von Nervensystemen von Lebewesen. Sie sind aber lediglich eine Abstraktion der Informationverarbeitung und versuchen nicht eine moeglichst genaue biologische Abbildung darzustellen. Es gibt nicht nur eine Art von Neuralem Netz, sondern es exisitieren die verschiedensten Architekturen, welche je nach Problemstellung ausgewaehlt werden muessen. Diese Arbeit wird sich spaeter vorallem mit sogennanten Autoencodern auseinandersetzen.

4.4.1 Perzeptron

Um den Aufbau und die Funktion eines Kuenstlichen Neuronalen Netz besser zu verstehen, wird im folgenden ein Vorgaenger des KNN erklaert: das **Perzeptron**.

Das einlagige Perzeptron wurde erstmals 1958 von Frank Rosenblatt vorgestellt. Dieses besteht aus einem einzigen Kuenstlichen Neuron. Dieses Kuenstliche Neuron hat mehrere binaere Inputs und einen einzigen binaeren Output. Binaer bedeutet, dass der Wert nur entweder 0 (*aus*) oder 1 (*ein*) sein kann. Des weiteren besitzt es mehrere sogenannte **Gewichte** $w_1, \ldots, w_n \in \mathbb{R}$, fuer jeden Input ein Gewicht. Diese sind reelle Zahlen, welche das Verhalten des Perzeptron bestimmen. Die **gewichtete Summe**, also die Summe aller Produkte der Inputs mit ihrem Gewicht, wird mit $\tilde{z}$ bezeichnet. Sie ist das gleiche wie das Skalarprodukt des Gewichtevektor mit dem Inputvektor: $\tilde{z} = \sum_{i=1}^{n} w_i x_i = \mathbf{w} \cdot \mathbf{x}$
Zusaetzlich besitzt das Perzeptron einen **Schwellenwert** $\tilde{b}$. Zusammen mit den Gewichten, bildet er die Modellparameter. Das Perzeptron verhaelt sich so, dass falls die gewichtete Summe groesser als der Schwellwert ist, das Neuron feuert, d.h. der Output betraegt 1. Andernfalls ist er 0. Siehe erster Teil der Gleichung (6). Es ist gaengig die Ungleichung der Bedingung in die Nullstellenform zu bringen und $\tilde{b}$ durch die **Neigung** (engl.: Bias) $b = -\tilde{b}$ zu ersetzten. Somit lautet die Ungleichung: $\tilde{z} + b > 0$. Der Term $\tilde{z} + b$ wird mit z bezeichnet. Siehe Rest der Gleichung (6). Die Neigung gibt an wie stark das Neuron dazu neigt zu feuern. Ein hohes b laesst ein Neuron auch trotz einigen Nullen in den Inputs feuern, waehrend es fuer ein tiefes b nicht feuern lassen wuerde.

$$p(\mathbf{x}) = \begin{cases} 1 & \text{falls } \tilde{z} > \tilde{b} \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \tilde{z} + b > 0 \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \mathbf{w} \cdot \mathbf{x} + b > 0 \\ 0 & \text{ansonsten} \end{cases} \tag{6}$$

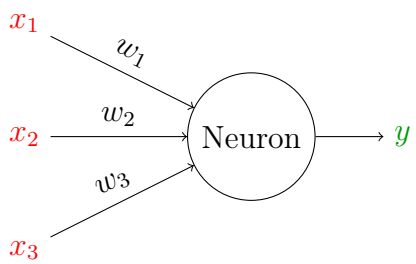

Abbildung 5: Perzeptron mit drei Inputs

Fuer das Trainieren des Perzeptron existieren spezielle Verfahren, welche hier aber nicht relevant sind. Das Gradientenverfahren kann naemlich nicht verwendet werden. Der Grund dafuer sollte spaeter in Sektion (4.4.3) einleuchtend werden.

Nun stellt sich die Frage, was ein Perzeptron erlernen kann und wofuer es genutzt werden kann. Das Perzeptron ist lediglich ein **Linearer Klassifikator** der Form $y = w_1x_1 + \cdots + w_nx_n$. Es kann die Features in zwei Klassen 0 oder 1 einordnen. Ueberschreitet y den Schwellenwert, werden die Features der Klasse 1 zugeordnet, sonst der Klasse 0. Jedoch muessen diese Klassen linear separierbar sein. Das bedeutet, dass die Inputvektoren $\mathbf{x}$ durch eine Hyperebene trennbar sein muessen. Bei zwei Features (zwei Inputs) handelt es sich bei der Hyperebene um eine Gerade welche die Vektoren auftrennt. Siehe Abbildung (6). Fuer drei Featuers ist es eine Ebene. ZU ABSTRAKT -> BEISPIEL
Eine Funktionen, welche nicht linear separierbar ist (z.B XOR-Verknuepfungen), kann von einem Perzeptron nicht erlernt werden.

Abbildung 6: erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts)

[Mehrere, *Perzeptron — Wikipedia, Die freie Enyklopädie*]

4.4.2 Biologische Analogie

SCHREIBEN

4.4.3 Erweiterung der Kuenstlichen Neuronen

Ein Perzeptron ist, wie vorhin erklaert, nur in der Lage, lineare Klassifikationen durchzufuehren. Um nun auch kompliziertere Probleme zu loesen, muss das Prinzip ausgebaut werden. Ausserdem brauchen wir ein Kuenstliches Neuron, welches sich besonders gut als Baustein fuer KNNs eignet.

4.4.3.1 Kuenstliche Neuronen im Allgemeinen

Kuenstliche Neuronen sind immer so aufgebaut, dass sie einen oder mehrere Inputs haben und einen einzigen Output. Zu jedem Input x_i ist ein Gewicht w_{ji} assoziert. Zuerst wird die gewichtete Summe der Inputs $\tilde{z}$ gebildet. Die Neigung b wird ebenfalls draufaddiert, um z zu erhalten. Nun muss die sogennante Aktivierung a gebildet werden. Sie entspricht dem Output des Neurons. Die Aktivierung a ist das Resultat der Aktivierungsfunktion φ angewendet auf z. $a = \varphi(z)$. Die verschiedenen Kuenstlichen Neuronen unterscheiden sich fast nur in ihrer Aktivierungsfunktion.

4.4.3.2 Perzeptronen als Kuenstliche Neuronen

Zuerst noch einmal ein Blick auf das Perzeptron in Betracht der Aktivierungsfunktion. Ein wesentlicher Unterschied des Perzeptron gegenueber sonstigen Kuenstlichen Neuronen, besteht darin, dass seine Inputs und Outputs nur binaere Werte annehmen koennen. Um dieses Verhalten des Perzeptron zu erhalten, muss eine Stufenfunktion als Aktivierungfunktion verwendet werden: die Heaviside-Funktion Θ. Sie hat einen einzigen Stufensprung bei $x = 0$ vom Wert 0 auf 1. Siehe Abb. (7).

$$\varphi^{\text{hlim}}(z) = \Theta(z) = \begin{cases} 1 & \text{falls } z \geq 0 \\ 0 & \text{falls } z < 0 \end{cases}$$

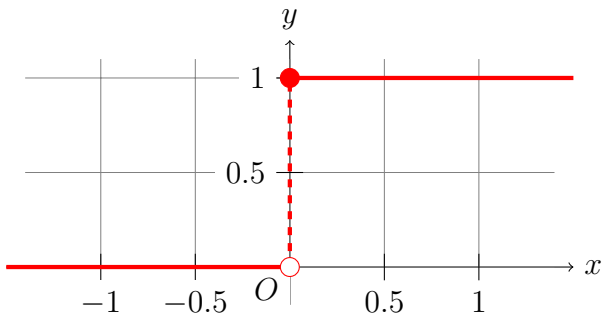

Abbildung 7: Definition und Graph der Heaviside-Funktion Θ

4.4.3.3 Stueckweise lineare Neuronen

Der naechste Schritt nach dem binaeren Perzeptron sind sogenannte stueckweise lineare Neuronen. Sie verwenden eine stueckweise lineare Aktivierungsfunktion. Diese bildet ein beschraenktes Intervall linear ab. Die Werte ausserhalb werden auf die konstanten Werte 0 oder 1 abbgebildet. Siehe Abb. (8).

Die Inputs koennen jetzt beliebige reelle Zahlen sein (vorzugsweise in der Naehe von 0 und 1). Ein KNN aus ausschliesslich linearen Neuronen ist ziemlich nutzlos, da jedliche Verkettung von linearen Funktion auch durch eine einzige lineare Funktion dargestellt werden koennte. Somit hat ein KNN gegenueber einem einzelnen Neuron keinen Mehrwert. Ausserdem koennen mit diesen Neuron auch nur lineare Probleme geloest werden.

$$\varphi^{\text{pwl}}(z) = \begin{cases} 1 & \text{falls } z > \frac{1}{2} \\ z + \frac{1}{2} & \text{falls } -\frac{1}{2} < z < \frac{1}{2} \\ 0 & \text{falls } z < \frac{1}{2} \end{cases}$$

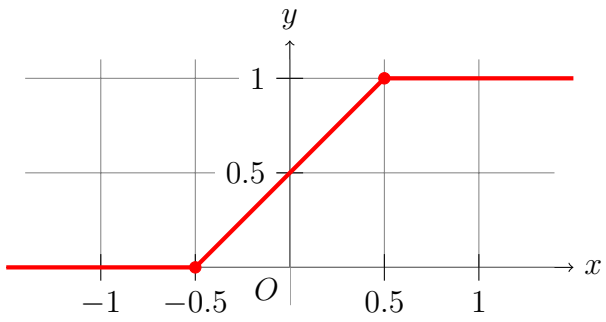

Abbildung 8: Formel und Graph einer stueckweisen linearen Funktion

4.4.3.4 Sigmoide Neuronen

Die logische naechste Erweiterung sind nicht-lineare Neuronen. Ein sehr beliebter Kandidat dafuer sind Sigmoide Neuronen. Den Namen haben sie von ihrer Aktivierungsfunktion: der Sigmoidfunktion σ. Eine sehr wichtige Eigenschaft von ihr ist, dass sie - im Gegensatz zu den vorhin genannten Aktivierungsfunktion - ueberall differenzierbar und strikt monoton steigend ist. Erst fuer diese Aktivierungsfunktion, kann das Gradientenverfahren angewendet werden und somit das KNN trainiert werden. Die anderen Funktionen haben Stellen bei welchen die Ableitungsfunktion null betraegt, somit kann das lokale Minimum nicht gefunden werden. Nicht nur die Sigmoidfunktion erfuellt diese Bedingungen, sondern auch andere. Die Sigmoidfunktion wird bevorzugt, da ihre Ableitung sehr simpel ist, siehe Abb. (9). Die Nicht-linearitaet dieser Neuronen ermoeglicht ein Erlernen von deutlich komplexeren Sachverhalten. Und wie spaeter in Sektion (4.4.8) erlautert, ist es moeglich mit der Kompostion von nicht-linearen Funktionen jede belibige Funktion zu approximieren!

Die Simgmoid-Funktion besitzt eine einzige Wendestelle bei $x = 0$ und hat zwei Asymptoten, eine fuer $x \to -\infty$ mit $y = 0$ und eine zweite fuer $x \to \infty$ mit $y = 1$. Siehe Abb. (9).

$$\varphi^{\text{sig}}(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

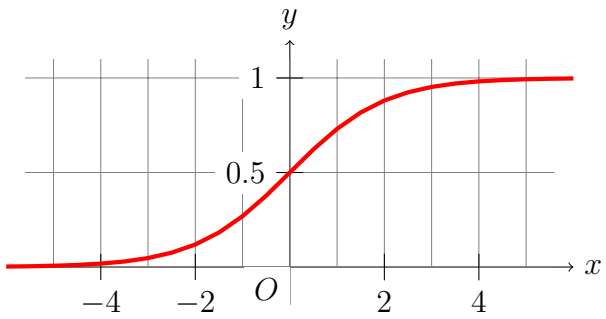

Abbildung 9: Definition, Ableitung und Graph der Sigmoid-Funktion σ

4.4.4 Topologie der Kuenstlichen Neuronalen Netzen

Nun sollten diese Sigmoiden-Neuronen als Bausteine verwendet werden, um ein Kuenstliches Neuronales Netz zu bilden. Dazu werden sie miteinander verbundet und bilden so ein Netz, aehnlich wie ein Nervensystem. Diese Neuronen sind in verschieden Schichten (engl.: Layers) arangiert. Die erste ist die **Inputschicht**. Sie behinhalten die Inputneuronen. Dies sind eigentlich keine richtigen Neuronen sondern eher Platzhalter fuer ihren jeweiligen Inputwert x_i. Als letztes kommt die **Outputschicht** mit den Outputneuronen, welche jeweils einen Outputwert y_i besitzen. Dazwischen liegen die **Zwischenschichten** (engl.: Hiddenlayer). Von ihnen kann es beliebig viele geben und in ihnen beliebig viele Neuronen. Der Aufbau eines KNN bezeichnet man als **Topologie** des Netzes. Die Topologie umfasst viele Hyperparameter. Darunter sind zum Beispiel die Anzahl Zwischenschichten, wie auch die Anzahl Neuronen pro Schicht.

Jedes Neuron aus einer Schicht ist mit jedem Neuron aus der naechsten Schicht ueber Verbindungen gekoppelt. Alle Verbindungen besitzten ein Gewicht analog zu den Inputs des Perzeptron. Die Aktivierung, also der Output, eines Neurons wandert entlang den jeweiligen Verbindung zu allen Neuronen der naechsten Schicht und dient als deren Input. Die soeben beschriebe Art von KNN nennt man **Feedforward-Netz**, da alle Werte ausschliesslich nach vorne propagiert werden.
In Abbildung (4.4.5) ist ein Beispiel eines Neuronalen Netzes abgebildet. In diesem Fall besitz es sowohl 4 Inputs, wie auch 4 Outputs. Es hat ausserdem 3 Zwischenschichten. Die zwei aueserren Zwischenschichten haben 3 Neuronen und die dazwischen hat sogar 4 Neuronen.
ABBILDUNG MIT BESCHRIFTUNG DER GEWICHTE

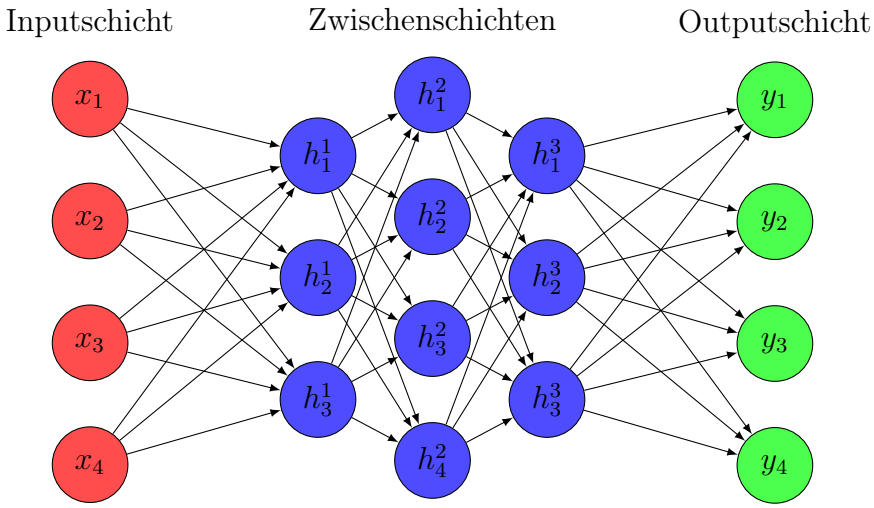

Abbildung 10: Schichten eines KNNs

4.4.5 Vorwaertspropagierung

Jetzt, da der Aufbau eines KNNs erklaert wurde, sollte nun die mathematische Funktionsweise des Modells erklaert werden. Hierfuer muessen einige Konventionen zur Bezeichnung der Teile eines KNNs getroffen werden:

- n_j^l ist das j-te Neuron in der l-ten Schicht.
- z_j^l ist die gewichtete Summe der Inputs des j-ten Neuron in der l-ten Schicht.
- a_j^l ist die Aktivierung des j-ten Neurons in der l-ten Schicht.
- b_j^l ist die Neigung des j-ten Neuron in der l-ten Schicht.
- $w_{j,k}^l$ ist das Gewicht der Verbindung vom k-ten Neuron in der $(l-1)$-ten Schicht zum j-ten Neuron in der l-ten Schicht (man beachte die Reihenfolge) [2].
- L soll die gesamte Anzahl der Schichten sein.
- N_l ist die Anzahl Neuronen in der l-ten Schicht.
- φ ist die gewaehlte Aktivierungsfunktion (grundsaetzlich ist diese immer die Sigmoidfunktion σ).

ABBILDUNG MIT GEWICHTEBESCHRIFTUNGEN (KORREKT!)

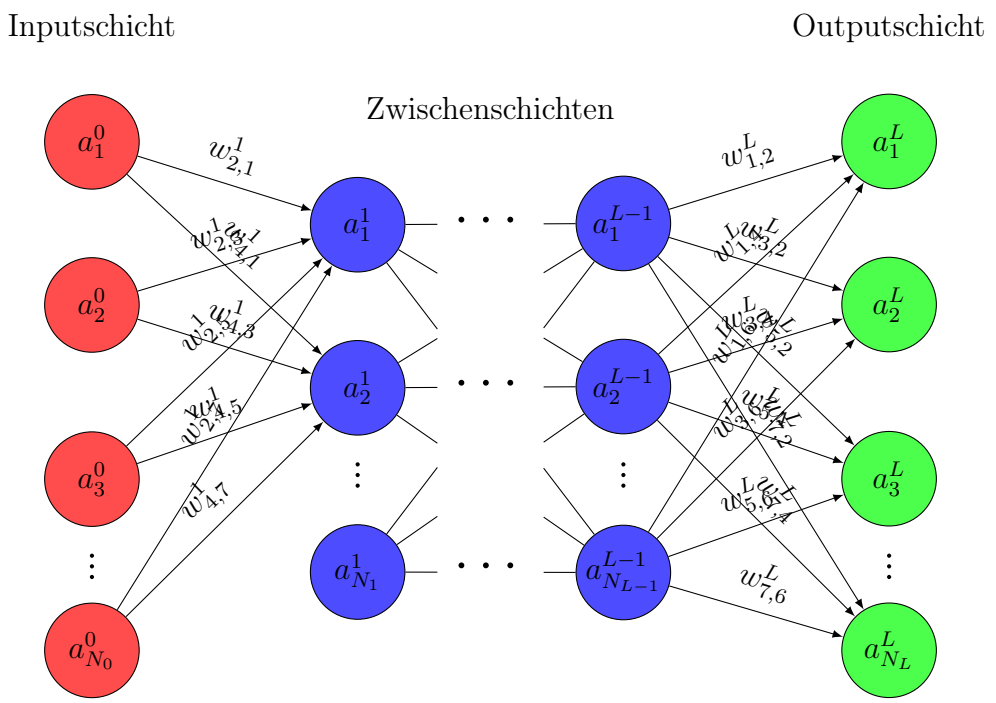

Abbildung 11: zum Verstaentniss der Nomenklatur

Die Vorwaertspropagierung beginnt bei den Inputneuronen, welche jeweils einen Inputwert in sich tragen. Diese werden, um fuer eine kohaerente Nomenklatur zu sorgen, gleich wie die Aktivierungen der anderen Neuronen mit a_j^0 bezeichnet, wobei j der Index des Neurons ist.

[2] Diese Konvention scheint auf den ersten Blick unintuitiv, macht jedoch Sinn fuer die spaeteren mathematischen Operationen mit Matrizen in Sektion (4.4.7).

Nun muessen die restlichen Aktivierungen der Neuronen berechnet werden. Dies geschieht rekursiv, anhand der Aktivierungen der voherigen Schicht. Und zwar folgendermassen (ersichtlich in Gleichung (7)):
Zuerst laueft eine Summe ueber alle Neuronen n_k^{l-1} der voherigen Schicht $(l-1)$. Dabei wird die gewichtete Summe der Aktivierungen a_k^{l-1} mit den assozierten Gewichten $w_{j,k}^l$ gebildet. Hierbei ist das Gewicht jenes, welches das k-te Neuron der $(l-1)$-ten Schicht mit dem j-ten Neuron der l-ten Schicht verbindet. Zusaetzlich gehoert zu der gewichteten Summe auch die jeweilige Neigung b_j^l, welche dazuaddiert wird. Diese gewichtete Summe wird mit z_j^l bezeichnet.

$$z_j^l = \sum_{k=1}^{N_l} w_{j,k}^l a_k^{l-1} + b_j^l \tag{7}$$

Auf diese Summe wird dann die Aktivierungsfunktion φ angewandt. Das ist dann die Aktivierung a_j^l des Neurons.

$$a_j^l = \varphi\left(\sum_{k=1}^{N_l} w_{j,k}^l a_k^{l-1} + b_j^l\right) = \varphi\left(z_j^l\right) \tag{8}$$

Da KNNs weit verbreitet sind in der Bilderkennung und Spracherkennung, ist es nicht unueblich, dass diese sehr viele Neuronen und Verbindungen (ueber 100'000) besitzen. Um hierbei nicht den Ueberblick zu verlieren und um nicht in den Indizes zu ertrinken, greift man auf **Lineare Algebra** zurueck. Man verwendet Matrizen und Vektoren um die vielen Variabeln zusammen zu fassen. Ausserdem besteht ein weiterer Vorteil darin, dass Computer mithilfe von Vektor- und Matrixoperationen die Berechnungen parallisieren koennen und in kuerzerer Zeit und mit weniger Ressourcen viele Brechnungen gleichzeitig ausfuehren koennen. Dies beschleunigt das Training der Modelle um ein Vielfaches. Dies wird spaeter in Sektion (**??**) noch weiter thematisiert.

Die Inputs $\mathbf{x}$, Outputs $\mathbf{y}$ und Labels $\hat{\mathbf{y}}$ haben wir schon von Anfang an als Vektoren geschrieben. Nun sollen noch die Parameter und die restlichen Komponenten eines KNNs als Vektoren und Matrizen geschrieben werden. Sowohl alle gewichteten Summen z_j^l, wie auch alle Aktivierungen a_j^l einer Schicht l, werden in Vektoren $\mathbf{z}^l$ und $\mathbf{a}^l$ zusammengefasst. Alle Neigung b_j^l einer Schicht l bildet ebenfalls eines Vektor $\mathbf{b}^l$. Zu guter letzt, wird noch eine **Gewichtsmatrix** $\boldsymbol{W}^l$ fuer jede Schicht l definiert. Die Eintraege dieser Matrix sind einfach die Gewichte welche *zu* den Neuronen der l-ten Schicht verlaufen. Das heisst der Eintrag in der j-ten Zeile und in der k-ten Spalte ist $w_{j,k}^l$ (Siehe Gl. (9)).

$$\boldsymbol{W}^l = \begin{pmatrix} w_{1,1}^l & w_{1,2}^l & \cdots & w_{1,n}^l \\ w_{2,1}^l & w_{2,2}^l & \cdots & w_{2,n}^l \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1}^l & w_{m,2}^l & \cdots & w_{m,n}^l \end{pmatrix} \tag{9}$$

Mit diesen Definitionen kann nun Gleichung (8) in Matrixform geschrieben werden. Hierfuer muss noch ein weiteres mathematisches Konzept eingefuehrt werden: die Vektorisierung einer Funktion.

Vektorisierung einer Funktion

Die Vektorisierung einer Funktion f, geschrieben als $f[\mathbf{v}]$ ist eine neue Funktion, welche als Rueckgabewert einen Vektor hat, auf dessen Komponenten jeweils immer die Funktion f angewendet wird.

$$f[\mathbf{v}] = \begin{pmatrix} f(v_1) \\ \vdots \\ f(v_n) \end{pmatrix}$$

Nun kann die Matrixschreibweise eingefuehrt werden (siehe Gleichung (11)).
Zuerst wird eine Matrixmultiplikation der Gewichtsmatrix $\boldsymbol{W}^l$ mit dem Aktivierungsvektor $\mathbf{a}^{l-1}$ der vorherigen Schicht durchgefuehrt. Darauf wird der Neigunsvektor addiert. Und zu guter Letzt wird die vektorisierte Aktivierungsfunktion φ angewandt.

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l \tag{10}$$

$$\mathbf{a}^l = \varphi\left[\boldsymbol{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l\right] = \varphi\left[\mathbf{z}^l\right] \tag{11}$$

4.4.6 Gewichte und Neigungen initalisieren*

4.4.7 Rueckwaertspropagierung

Die wahre Herausforderung besteht beim Gradientenverfahren darin, die partiellen Ableitungen der Modellparameter, also die Komponenten des Gradienten, zu bestimmen. Anderst gesamt muessen alle $(\partial C)/(\partial w^l_{j,k})$ wie auch alle $(\partial C)/(\partial b^l_k)$ berechnet werden. Das Verfahren zum bestimmen dieser Ausdruecke ist so spezfisch und aufwendig, dass das Gradientenverfahren fuer KNNs einen eigen Namen hat: die **Rueckwaertspropagierung** (engl.: Backpropagation).
Das Ziel dieses Algorithmus ist es die beiden Partiellen Ableitungen $(\partial C)/(\partial w^l_{j,k})$ und $(\partial C)/(\partial b^l_j)$ zu bestimmen. Hierfuer ist es sinnvoll eine Zwischenwert, welchen wir Fehler δ^l_j nennen.

Hadamard-Produkt

Das Hadamard-Produkt ist ein spezielles Produkt zweier gleichgrossen Matrizen. Die resultierende Matrix ergibt sich aus der elementweisen Multiplikation der Ausgangsmatrizen.

$$\boldsymbol{A} \odot \boldsymbol{B} = \begin{pmatrix} a_{1,1}b_{1,1} & \cdots & a_{1,n}b_{1,n} \\ \vdots & \ddots & \vdots \\ a_{m,1}b_{m,1} & \cdots & a_{m,n}b_{m,n} \end{pmatrix} \in \mathbb{R}^{m \times n} \qquad \mathbf{v} \odot \mathbf{w} = \begin{pmatrix} v_1 w_1 \\ \vdots \\ v_n w_n \end{pmatrix}$$

$$\delta^l_j := \frac{\partial C}{\partial z^l_j} \tag{12}$$

Viel Kettenregel.

$$\frac{\partial C}{\partial w_{j,k}} = \frac{\partial C}{\partial a_k} \frac{\partial a_k}{\partial z_k} \frac{\partial z_k}{\partial w_{j,k}} \tag{13}$$

Die vier magischen Gleichungen:
Der Fehler in der Outputschicht

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L) \tag{BP1}$$

Matrixform

$$\delta^L = \nabla_{\mathbf{a}} C \odot \sigma'(z^L) \tag{BP1a}$$

Das tiefgestellte $_a$ des Nablas, gibt an, dass es sich um den Gradienten mit den partiellen Ableitungen bezueglich den Aktivierungen a handelt.
Fehler anhand des Fehlers der naechsten Schicht

$$\delta^l = ((w^{l+1})^{\mathrm{T}} \delta^{l+1}) \odot \sigma'(z^l) \tag{BP2}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{BP3}$$

$$\frac{\partial C}{\partial w_{j,k}^l} = a_k^{l-1} \delta_j^l \tag{BP4}$$

4.4.8 Universal Approximation Theorem

Neural Network kann alles erlernen

4.5 Convolutional Neural Networks

Viele Anwendungen von Machine Learning sind verbunden mit Bild- oder Audioverarbeitung, wie z.B Bildklassifizierung, Gesichtserkennung oder Spracherkennung. Vorallem aber fuer hochaufloesende Bilder sind die KNNs, die wir soeben kennengelernt haben, nicht geeignet. Sie sind zum Teil gar nicht in der Lage eine Korrelation zwischen den Inputs und Outputs zu erlernen. Um diesen Umstand zu erklaeren, wird nun ein kleines Beispielmodell erlauetert:

Es soll ein KNN designed werden, welches einer Photographie danach klassifizieren soll, ob darauf ein Hund sichtbar ist oder nicht. Wir nehmen fuer dieses Beispiel bereits ein relativ niedrig aufgeloestes Bild mit 256×256 Pixel, dies entspricht weniger als 0.07 Megapixel (ein iPhone XS hat eine Kamera mit 12 Megapixel). Um die verschiedenen Farben zu codieren besitzt jeder Pixel drei Komponten R, G und B. Somit hat dieses Bild insgesamt $256 \times 256 \times 3 = 196'608$ Komponenten. Jede Komponente ist ein Feature welches das KNN zu verrechnen hat. Somit bestuende die erste Schicht des Netzwerkes aus fast $200'000$ Neuronen. Um diese Schicht nun mit seiner Nachbarsschicht, welche gleiche Dimensionen besitzt, zu verbinden, braucht es $196'608 \times 196'608 = 38'654'705'664$ Verbindungen und damit gleich so viele Gewichte! Fuer ein Netwerke ohne eine einzige Zwischenschichten gaebe es also ueber 38 Milliarden Modellparameter zu erlernen! Das dies nicht realistisch ist, sollte auf der Hand liegen.

Nun sollte klar sein, dass eine andere Modellarchitektur noetig ist, um Machine Learning auf Bilder anzuwenden. Fuer genau solche Anwendungen wurde eine modifizierte Version eines KNNs entwickelt: das **Convolutional Neural Network** (CNN). Diese Art von Netzwerk, macht Gebrauch von Konzepten aus der klassischen Bildverarbeitung, wie sie mit beispielsweise Photoshop gemacht werden kann. Wie beim Perzeptron und bei den klassischen KNNs auch, ist die Architektur biologisch inspiriert. Der folgende Abschnitt wird die Funktionsweise eines solchen CNNs erklaeren.

[deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*]

4.5.1 Bilder als 3D-Matrizen

Bei der Betrachtung von Bildern fuer Machine Learning, ist es sinnvoll, diese nicht als eine Anordnung von verschieden farbigen Pixeln zu interpretieren, sondern sie als **3D-Matrizen** zu untersuchen.

3D-Matrix

Eine 3D-Matrix ist ein mathematischen Objekt und eine Erweiterung von Matrizen auf 3-dimensionale Zahlenanordnung. Vorstellen kann man sich eine 3D-Matrix als ein Volumen dass Zahlen in einem Raster angeordnet hat. Die Zahlen sind dabei die Elemente der Matrix. Analog zum Volumen, bezeichnet man die Form der Matrix mit Hoehe, Breite und Tiefe. Eine Matrix von der Form $3 \times 3 \times 3$ koennte folgendermassen aussehen:
TIKZPICTURE EINER 3D-Matrix

Ein Bild kann somit einfach als 3D-Matrix, bezeichnet mit $\boldsymbol{B}_{h \times w \times c}$ der Form (Bildhoehe $\times$ Bildbreite $\times$ Anzahl Farbkomponenten) betrachtet werden. Die Elemente der Matrix nehmen dann einfach die Werte der Pixelkomponenten an. Ein schwarzweiss Bild hat nur eine Komponente, welche die Helligkeit angibt. Somit waere es eine normale 2D-Matrix $\boldsymbol{B}_{h \times w}$. Siehe Abb. (12).

Eine solche **Bildmatrix** stellt dann den Input $\boldsymbol{X}$ eines CNNs dar und nicht mehr wie voher ein Vektor $\mathbf{x}$.

Abbildung 12: Beispiel einer Bildmatrix

Abbildung 13: 2-dimensionalen Schicht von Neuronen

4.5.2 Filter

4.5.2.1 Filter in der Bildverarbeitung

Filter (engl. auch: Kernel) sind in der Bildverarbeitung sehr verbreitet. Jeder kennt sie entweder von Photoshop, von Instagram oder sonstigen Bildbearbeitungsprogrammen. Auch CNNs machen Gebrauch von solchen Filtern, in diesem Fall um die Features eines Bildes zu erlernen. Aus diktaktischen Gruenden betrachten wir zuerst nur 2D-Filter, welche sich fuer graustufige Bilder $\boldsymbol{B}_{h\times w}$ eignen.

Abbildung 14: der Instagramfilter: Blablabla

Filter sind einfach Regionen, deutlich kleiner als das verarbeitete Bild, welche ueber alle Pixel wandert und diese manipuliert und so wieder ein neues Bild $\boldsymbol{B}^*$ liefern. Mathematisch gesehen handelt es sich bei einem Filter einfach um eine Matrix, die sogennante **Filtermatrix** oder Faltungsmatrix (siehe Abb. (15)), bezeichnet mit $\boldsymbol{F}_{f\times f}$. Solche Filter sind immer quadratisch, wobei ihre Zeilen- bzw. Spaltenlaenge mit f bezeichnet wird. Dabei ist f immer eine ungerade Zahl, damit ein Element, das sogennante **Zentralelement**, immer im Zenturm des Filters liegt.

Abbildung 15: eine Filtermatrix

Das Verhalten des Filters wird durch seine Matrixeintraege bestimmt. Die Elemente koennen also so gewaehlt werden, dass wenn der Filter auf ein Bild angewandt wird, er bestimmte Features aus dem Bild hervorhebt. Solche Features koennten zum Beispiel Raender oder Kanten sein, welche betont werden.

Ein gutes Beispiel ist der Sobel-Filter. Er ist ein typischer Kantendetektions-Filter mit einer Groesse von $f = 3$. Wir betrachten nun den Sobelfilter $\boldsymbol{G}_x$, welcher horizontale Kanten akzentuiert. Die Eintraege dieses Filters lauten wie folgt:

$$\boldsymbol{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Angewandt auf ein Bild, erhaelt man ein neues Bild, bei welchem alle Bereiche schwarz sind und die erkannten Kanten weiss eingefaerbt sind. Der Filter erkennt dabei jede Stelle als Kante, welche zwei Regionen mit genug grossem Kontrast voneinander trennt. In Abbildung (16) wurde der horizontale Sobelfilter auf ein Beispielbild angewandt. Links ist das Ursprungsbild und rechts ist das prozessierte Bild.

Abbildung 16: Sobel-Filter angewandt auf Beispielsbild

[Mehrere, *Sobel operator — Wikipedia, The Free Encyclopedia*]
[deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*]

4.5.2.2 Biologische Motivation?

Jedoch ist zu erkennen, dass Bilddaten sich deutlich von sonstigen Daten unterscheiden. Auch unser Organismus hat diese Tatsache erkannt und deshalb spezielle Zellen ausgebildet (sogennante Ganglienzellen), welche sich mit der Vorverarbeitung von visuellen Informationen beschaeftigt. Diese Zellen und weitere fuehren auch eine Art von Kantendetektion durch mithilfe von Kontrastverstaerkung.

[al., *Klett Markl Biologie Oberstufe*]

4.5.2.3 Filteroperationen als Faltungen

Wie wendet man nun einen solchen Filter auf ein Bild an? Um das Prozedere einer Filteroperation leichter verstaendlich zu machen, wird sie erneut fuer ein graustufiges Bild $\boldsymbol{B}_{h\times w}$ erlauetert. Spaeter werden auch noch farbige Bilder erklaert.

Grundsaetzlich handelt es sich bei einer Anwendung eines Filters auf ein Bild um eine spezifische mathematische Operation: eine **Faltung** (engl.: **Convolution**). [3] Daher ruehrt auch der Name des Convolutional Neural Networks. Wir werden die Faltung als mathematische Operation nicht im vollen Umfang betrachten, da sie relativ kompliziert ist. Wir werden uns lediglich auf die Bedeutung der Faltung fuer Filter beschraenken.

Eine Filteroperation ist nichts anderes als eine Filtermatrix $\boldsymbol{F}$, welche ueber eine Bildmatrix $\boldsymbol{B}$ gefaltet wird und somit eine eine neue Bildmatrix $\boldsymbol{B}^*$ bildet. Man schreibt: $\boldsymbol{B}^* = \boldsymbol{F} * \boldsymbol{B}$.
Um nun diese Faltung zu verstehen, verwenden wir als Beispiel eine 2D-Filtermatrix $\boldsymbol{F}_{3\times 3}$, bei welcher $f = 3$ gewaehlt wurde. Wie bereits vorhin erwaehnt wandern der Filter ueber das Bild. Dabei befindet sich der Filter immer ueber einer Region des Bilder, welche gleich gross ist, wie der Filter selber (in diesem Fall (3×3)), so dass den Filtermatrixeintraegen immer eindeutig Bildmatrixeintraege entsprechen. Diese Region des Bildes bezeichnet man als das **Rezeptive Feld** des Filters (siehe Abb. (17)). Man schreibt dafuer $\tilde{\boldsymbol{B}}^{(y,x)}$, wobei die hochgestellten Indizes (y, x) die Position des Zentralelements im rezeptiven Feldes angiebt. Somit ist das rezeptiven Feld einfach eine Untermatrix $\tilde{\boldsymbol{B}}_{3\times 3}$ des Obermatrix $\boldsymbol{B}$.

Abbildung 17: ein Filter und sein rezeptives Feld

Der Filter beginnt nun oben links ueber das Bild zu wandern. Somit hat der Filter zu beginn das rezeptive Feld $\tilde{\boldsymbol{B}}^{(2,2)}$. [4] Nun werden die Elemente der Filters mit den Elementen des rezeptiven Feldes verrechnet, welche die gleichen Dimensionen besitzen. Dabei wird einfach das Hadamard-Produkt (das elementweise Produkt) des beiden Matrizen miteinander gebildet. Nun erhalten wir eine neue Matrix $\boldsymbol{P}_{3\times 3} = \boldsymbol{F} \odot \tilde{\boldsymbol{B}}^{(2,2)}$. Nun werden noch alle Elemente der neuen Matrix $\boldsymbol{P}$ aufsummiert. Diese Zahl ist dann der Grauwert des ersten Pixels $(\boldsymbol{B}^*)_{1,1}$ des bearbeiteten Bildes.

$$(\boldsymbol{B}^*)_{1,1} = \sum_{y=1}^{3} \sum_{x=1}^{3} (\boldsymbol{P})_{y,x}$$

Nun wird der Filter um ein Element nach rechts verschoben und die gleiche Prozedur angewendet, um den zweiten Pixel zu berechnen. Dies wird durchgezogen, bis eine ganze Zeile der

[3] Eigentlich ist die Filteroperation keine richtige Faltung im mathematischen Sinne, sondern eher eine sogennante Kreuzkorrelation. Dennoch wird im Verlauf dieser Arbeit der Begriff Faltung im Zusammenhang mit Filteroperationen verwendet.
Fuer Intressierte: Die Kreuzkorrelation funktioniert gleich wie eine Faltung, jedoch wird die eine Matrix zuerst horizontal, wie auch vertikal gespiegelt, bevor die eigentliche Faltung angewandt wird.

[4] Die Indizes lauten $(2, 2)$, da das Zentralelement sich dort befindet. Der Filter darf nunmal nicht ueber die Raender des Bildes hinaus gehen

Bildmatrix durchstreift wurde. Danach wird der Filter wieder nach ganz rechts verschoben und er bewegt sich ein Element nach unten. Das geschieht, bis das ganze Bild verrechnet wurde. Dabei werden die Elemente des urspruenglichen Bildes durchaus mehrmals verrechnet, da es zu einer Ueberlappung des vorherigen Filter und des verschobenen Filters kommt.

Abbildung 18: Schema, wie ein Filter ueber ein Bild laueft

Es ist zu bemerken, dass das neue Bild $\boldsymbol{B}^*_{(h-2)\times(w-2)}$ nicht mehr die gleichen Masse hat, wie das Ursprungsbild $\boldsymbol{B}_{h\times w}$. Das liegt daran, dass pro Lage des Filters jeweils ein Pixel des neuen Bildes berechnet wird. Man kann sich vorstellen, dass an der Position des Zentralelements jeweils ein neuer Pixel entsteht. Dabei muss der Filter immer eine vollstaendige Region als rezeptives Feld haben. Dadurch liegt auf den Raendern niemals das Zentralelement und diese fallen somit weg.

[deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*]

4.5.2.4 Allgemeine Filteroperation

Fuer farbige Bilder ist das Vorgehen praktisch das selbe. Es werden nun lediglich 3D-Matrizen anstatt 2D-Matizen verwendet. Wir betrachten die allgemeine Bildmatrix $\boldsymbol{B}_{h\times w\times c}$, wobei c die Farbkomponenten (engl.: channels) sind. Nun benutzt man Filter $\boldsymbol{F}_{f\times f\times c}$ mit beliebiger Groesse f aber mit der gleicher Tiefe c wie das Ursprungsbild $\boldsymbol{B}$.
Wikipedia tolle Beschreibung: Prozess der Aufsummierung aller elemente eines Bildes zu seinen lokalen Nachbarn, gewichtet durch den Filter

$$\boldsymbol{P}^{(y,x)}_{f\times f} = \boldsymbol{F}_{f\times f} \odot \tilde{\boldsymbol{B}}^{(y,x)}_{f\times f} \quad \forall y \in \{1,\dots,h\}, \forall y \in \{1,\dots,w\} \tag{14}$$

$$(\boldsymbol{B}^*_{(h-1)\times(w-1)})_{y,x} = \sum_{i=1}^{f}\sum_{j=1}^{f}(\boldsymbol{P}^{(y,x)})_{i,j} \tag{15}$$

ALLGEMEINSTE FILTER GLEICHUNGEN

4.5.2.5 Wieso Filter?

Es stellt sich nun die Frage, weshalb sich Filter fuer Machine Learning eignen. Wie bereits erwaehnt besteht die Aufgabe von Filtern darin bestimmte Features eines Bildes hervorzuheben und die restlichen auszublenden. Genau die gleiche Aufgabe erfuellen die Neuronen in einem KNNs. Auch sie sollen Features der Inputsdaten erlernen, wobei gewisse Neuronen auf gewisse Features reagieren.

Jedoch ist zu erkennen, dass Bilddaten sich deutlich von sonstigen Daten unterscheiden. Bildfeatures bzw. Pixel sind immer im Kontext von ihren Nachbarn wichtig. Denn ein Pixel ist erst eine Kante oder eine Ecke, wenn er zwei verschiedene Kontraste voneinander trennt. Oder um die Konturen eines Gegenstandes zu erkennen, muss man eine ganze Fuelle von Pixeln betrachten. Man bezeichnet dies als **lokalisierte Features**.

Ein weiterer Aspekt besteht darin, dass Bildausschnitt nicht immer gleich orientiert sind. Wenn man ein Gesicht auf einem Bild erkennen moechte, sollte es keine Rolle spielen, wo auf dem Bild es sich befindet, oder welche Groesse es hat. Es besteht der Wunsch nach **Invarianz**. Typische Invarianzen, welcher ein Filter erfuellt, sind: Translationinvarianz, Rotationsinvarianz und Helligkeitsinvarianz.

4.5.2.6 Filter in CNNs

So nun betrachten wir Filter fuer CNNs. Der springende Punkt ist hierbei, dass die Filtereintraege Modellparameter sind und erlernt werden. Da sie die gleiche Funktion wie Gewichte in einem KNN einnehmen, bezeichnet man die Matrixelemente auch mit w.

$$\boldsymbol{F} = \begin{pmatrix} w_{1,1} & \cdots & w_{1,f} \\ \vdots & \ddots & \vdots \\ w_{f,1} & \cdots & w_{f,f} \end{pmatrix}$$

Die groesse des Filters bzw. das rezeptive Feld ist hierbei ein Hyperparameter.

4.5.2.7 Padding

Wie bereits erwaehnt schrumpfen die Bilder (an den Raendern), wenn man ein Filter auf sie anwendet. Zur Erinnerung: Dies liegt daran, dass pro Lage des Zentralelement nur ein Pixel des neuen Bildes entsteht und das Zentralelement nunmal nicht auf allen Ursprungspixeln zu liegen kommt. Die Raender fallen so weg. Diese Groesse des Wegfalls haengt von der Filtergroesse f ab. Hierbei giltet die Formel fuer die neue Groesse des Bildes.

$$s_{neu} = s_{alt} - f + 1 \tag{16}$$

$$\boldsymbol{B}_{s \times s} * \boldsymbol{F}_{f \times f} = \boldsymbol{B}^{*}_{s-f+1 \times s-f+1} \tag{17}$$

Das Problem hierbei ist, dass nach einigen Faltungen das Bild extrem geschrumpft ist und im Grenzfall kleiner als der Filter wird. Das darf natuerlich nicht passieren. Ausserdem geht dabei Information an den Raender verloren, da es im Innern des Bildes zu viel Overlapping der Filterlagen kommt und die Raeder nur sehr wenig einbezogen werden.

Diese Probleme werden mit sogennantem **Padding** behoben. Padding ist ein Schritt, welcher vor der eigentlichen Faltung stattfindet. Dabei werden einfach zusatzliche Raender (Zeilen und Spalten) an das Ursprungsbild angebracht. Die Matrix wird an den Enden erweitert. Die neuen Elemente werden dabei auf den Wert 0 gesetzt.

Abbildung 19: Padding

Das Padding p ist eine Zahl, welche angibt wieviele Elemente an allen Raendern hinzugefuegt werden. Padding $p = 1$ bedeutet, dass an allen Kanten jeweils eine Reihe bzw. Spalte hinzugefuegt wird. Begrifflich unterscheidet man zwei Arten von Padding:

- valid-Padding: Es werden keine zuesaezlichen Elemente. f ist also 0.
- same-Padding: Es werden so viele Reihen und Spalten angebracht, dass sich die Groesse des Bildes bei der Faltung erhalten bleibt.

Um das Padding zu berechnen fuer same-Padding verwendet man folgende Formel: (Ursprungsformel: $m = n + 2p - f + 1$)

$$p = \frac{f-1}{2} \tag{18}$$

4.5.2.8 Stride

Bis jetzt haben wir bei den Filterfaltungen, den Filter pro Anwendung immer nur um einen Pixel verschoben. Das muss aber nicht so sein. Man kann den Filter sich auch mit einer anderen Schrittgroesse verschieben. Diese Eigenschaft bezeichnet man als **Stride** s. Falls $s = 2$ gewaehlt wird, bedeutet das, dass der Filter sich pro Matrixmultiplikation um zwei Elemente bewegt. Somit wurde eine Position uebersprungen. Zur Folge hat dies, dass das neue Bild $\boldsymbol{B}^*$ deutlich kleiner geworden ist, denn das Zentralelement uebspringt somit auch diese Felder und bildet so deutlich weniger Pixel.

Folgende Formel beschreibt die Dimensionen des neuen Bildes $\boldsymbol{B}^*$ unter beruecksichtigung der Filtergroesse f, dem Padding p und dem Stride s.

$$m = \frac{n + 2p - f}{s} + 1 \tag{19}$$

Falls m keine ganze Zahl ergibt, wird abgerundet.

4.5.2.9 Mehrere Filter

Eine Filteroperation ist nicht auf einen einzigen Filter beschraenkt. Es koennen mehrere Filter auf das gleiche Ausgangsbild angewandt werden und zusammen ein Endbild erzeugen.

Die Anzahl Filter bezeichnen wir mit c. Nun macht man pro Filter eine Faltung ueber das Ursprungsbild $\boldsymbol{B}$, wobei jede Faltung eine neue Matrix $\boldsymbol{B}_i^*$ liefert, wobei i der Index des Filters ist. All diese Matrizen sind zweidimensional, unabhaengig davon, wieviel Komponten das Ursprungsbild hatte. Aus diesem Grund koennen die einzelnen Matrizen $\boldsymbol{B}_i^*$ aufeinander gelegt werden und somit eine grosse 3D-Endmatrix $\boldsymbol{B}^*$ bilden. Hierfuer muessen alle Filter gleich gross sein.
Weil die Anzahl Filter der Tiefe c des Endbildes, welche wir sonst als die Komponenten des Bildes bezeichnet haben, entspricht, verwendet man dafuer die gleiche Variable.
Mathematisch ausgedrueckt, bedeutet dies:

$$\boldsymbol{B}_{n\times n\times c} * \boldsymbol{F}_{1;f\times f\times c} * \cdots * \boldsymbol{F}_{n;f\times f\times c} = \boldsymbol{B}_{(n-f+1)\times(n-f+1)\times n} \tag{20}$$

4.5.3 Convolutional Layer

Convolution wird angewandt. Auf Outputmatrix wird Bias (Skalar) draufaddiert (auf jedes Element der gleiche Bias) Jeder Filter hat seinen eigenen Bias Nicht-lineare Aktivierungsfunktion wird auf Matrix angewandt Nun werden die verschieden Outputmatrizen der Filter ausgestackt zu einem 3D-Tensor fertig!

4.5.4 Pooling Layer

Pooling reduziert die Dimensionalitaet der Daten, indem eine Gruppe von Neuronen der vorherigen Schicht zu einem einzigen Neuron der naechsten Schicht zusammengefasst wird.
Local Pooling 2x2 Feld wird zu einem Neuron zusammengefasst. MaxPooling -> behaelt einfach maximalen Wert des Feld bei und verwirft den Rest. Average Pooling berechnet das arithmethische Mittel des Feldes und verwednet dieses. MaxPooling ist am effektivsten.
Pooling: Komprimierung des Bildes. Durchschnitt von 2x2 Pixeln berchnen -> ein Pixel des gepoolten Bild. -> 75% Informationsreduktion .
Pooling als Filter betrachtet: MaxPooling mit 2x2 Feld. Filtergroesse $f = 2$ und stride $s = 2$. Dies sind die Hyperparamter des MaxPooling. Hat also zwar Hyperparameter aber keinerlei Modellparameter

Abbildung 20: Schema eines CNNs

MaxPooling koennte auch andere Filtergroesse oder Stride haben. Formeln fuer Outputgroesse gelten auch hier.
Bei 3D-Tensoren aendert sich die Depth nicht, da das Pooling auf jeden Channel einzeln angewandt wird.
Es gibt zwei Konventionen: Entweder bezeichnet man die Einheit von einem ConvLayer und einem PoolLayer zu einer Schicht zusammengefasst werden oder sie werden als seperate Schichten betrachtet...

4.5.5 Fully-connected Layer

4.5.6 Das Netzwerk und sein Aufbau

Ein klassisches CNN besteht aus folgenden drei Typen von Schichten: Convolutional Schichten, Pooling Schichten und den bereits bekannten fully-connected Schichten. Zuerst kommt immer eine oder mehrere Convolutional Schichten, direkt darauf folgt dann eine Pooling Schicht. Dieses Muster kann dann beliebig oft wiederholt werden, bis die gewuenschte Komplexitaet erreicht wurde. Bei Bedarf koennen vor oder hinter einem solchen Muster eine fully-connected Schicht eingefuegt werden.
Beschriftung: Inputbilder werden wie Aktivierungen mit a^l bezeichnet. Die Filter sind aehnlich zu den Gewichten und werden mit W^l bezeichnet. Und der Bias bleibt aehnlich zum alten Bias b.
Konventionen zur Beschriftung: $f^{[l]}$ ist Filter Groesse
$p^{[l]}$ ist das padding
s^l ist stride
Input-Dimension ist $n_H^{[l-1]} \times n_W^{[l-1]} \times n_C^{[l-1]}$
Output-Dimension ist $n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$

$n[l] = \frac{n^{[l-1]}+2p^{[l]}-f^{[l]}}{s^{[l]}} + 1$ jeweils fuer Hoehe und Breite

n_C^l ist Anzahl Filter

Filter-Dimension ist: $f^{[l]} \times f^{[l]} \times n_C^{[l-1]}$
Aktivierungen $a^{[l]}$ -> $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
Gewichte $w^{[l]}$ -> $f^{[l]} \times f^{[l]} \times n_C^{[l-1]} \times n_c^{[l]}$ letzter Term ist ja gleich der Anzahl Filter
Vektor Bias: $n_c^{[l]}$
SchichtIndizes der Filter,Padding und stride sind gleich den Indizes des Output(aktivierungen)
Flattening ist, wenn man einen Tensor zu einem Vektor umformt

4.5.7 Eigenschaften von CNNs

Parameter/Weight-Sharing
ERkennung von gewissen Features (Ecken und Kanten) Bewegungs-Invarianz (durch Translations-Invarianz), Helligkeit-Invarianz alles folgend von geteilten Gewichten
Somit reduziert man die Information aufs Wesentliche und vernachlaessigt die Noise. Geteilte Gewichte
Lokale Konnektivitaet
Weil nicht fully-connected -> Overfitting erschwaert / unmoeglich

4.6 Autoencoder

4.6.1 Autoencoder

Lernen effiziente Datenrepraesenationen. Komprimierung. Unsupervised. Dimensionalitaetsreduktion.
Mathematik:

$$\phi : \mathcal{X} \to \mathcal{F} \tag{21}$$
$$\psi : \mathcal{F} \to \mathcal{X} \tag{22}$$
$$\phi, \psi = \arg\min_{\phi,\psi} \parallel X - (\phi \circ \psi)X \parallel^2 \tag{23}$$

4.6.2 Bedeutung von Autoencodern

4.6.2.1 Datenkompression

Falls ein Autoencoder nur aus linearen Neuronen besteht, ist die Dimensionsreduktion sehr aehnlich zu der Hauptkomponentenanalyse.
Problem: Netzwerk versucht Durchschnitt aller Trainingsdaten zu erlernen.

4.6.2.2 Datengenerierung

4.6.2.3 Eigenschaften erkennen

4.6.3 Variational Autoencoder?

4.6.4 Detangled Autoencoder?

4.7 Programmiertools

4.7.1 Tensorflow

Tensor

Ein Tensor ist eine verallgemeinerung von Matrizen auf n-dimensionale geometrische Objekte. Vorstellen kann man sich einen Tensor als einen Hyperrechteck mit n Dimensionen, innerhalb dessen die Zahlen in einem Raster angeordnet sind. Diese Zahlen sind die Elemente des Tensors. Dabei bezeichnet n den sogennaten Rang oder die Stufe des Tensors. Ein Tensor nullten Ranges ist ein Skalar, einer erster Stufe ein Vektor und Rang 2 ist eine Matrix. Tensoren haben noch viel weitreichendere Bedeutungen, vorallem fuer multilineare Abbildungen. Diese sind aber hier nicht relevant.

Ein Tensor dritten Ranges von der Form $(3 \times 3 \times 3)$ kann man sich folgendermassen vorstellen:

$$\overline{\mathbf{T}} \nexists [$$

5 Entwicklung eines Autoencoders

5.1 Tools

5.1.0.1 Genutzte Tools

- Spacemacs (Emacs)
- Arch Linux
- Python
- Tensorflow (CUDA/CUDNN)
- Keras
- LaTeX

5.1.1 Python

5.1.2 Tensorflow

5.1.3 Keras

5.2 Idee

5.3 Voraussetzungen

5.3.1 Trainingsdaten

MegaFace Dataset

5.4 Programmieren

6 Diskussion?

7 Schlusswort

Literatur

[1] Sven Gemballa et al. *Klett Markl Biologie Oberstufe*. Juergen Markl, 2010.

[2] Andrew NG - deeplearning.ai. *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*. [Online; besucht am 17. Juni 2019]. 2017. URL: https://www.youtube.com/playlist?list=PLkDaE6sCZn6Gl29AoE31iwdVwSG-KnDzF.

[3] Mehrere. *Perzeptron — Wikipedia, Die freie Enyklopädie*. [Online; besucht am 28. April 2019]. 2019. URL: https://de.wikipedia.org/wiki/Perzeptron.

[4] Mehrere. *Sobel operator — Wikipedia, The Free Encyclopedia*. [Online; besucht am 17. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Sobel_operator.

Abbildungsverzeichnis

1 das beschriebene Machine Learning Modells . . . 7
2 beschriebenes Beispielmodells . . . 8
3 Visualisierung des Gradientenabstieg . . . 10
4 Auswirkungen verschiedener Lernraten . . . 11
5 Perzeptron mit drei Inputs . . . 13
6 erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts) . . . 14
7 Definition und Graph der Heaviside-Funktion Θ . . . 15
8 Formel und Graph einer stueckweisen linearen Funktion . . . 15
9 Definition, Ableitung und Graph der Sigmoid-Funktion σ . . . 16
10 Schichten eines KNNs . . . 16
11 zum Verstaentniss der Nomenklatur . . . 17
12 Beispiel einer Bildmatrix . . . 22
13 2-dimensionalen Schicht von Neuronen . . . 22
14 der Instagramfilter: Blablabla . . . 22
15 eine Filtermatrix . . . 22
16 Sobel-Filter angewandt auf Beispielsbild . . . 22
17 ein Filter und sein rezeptives Feld . . . 23
18 Schema, wie ein Filter ueber ein Bild laueft . . . 24
19 Padding . . . 25
20 Schema eines CNNs . . . 27

Tabellenverzeichnis

Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich diese Arbeit selbständig durchgeführt und keine anderen als die angegebene Quellen, Hilfsmittel und Hilfspersonen beigezogen habe. Alle Textstellen in der Arbeit, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet.

Luis Wirth